

hwk2/tb1_data_transfer.vhd

```

1 -----
2 -- TB1_DataTransfer -- SH-2 CPU Integration Testbench 1
3 --
4 -- Two phases controlled by a single signal "loading":
5 --   loading='1' : TB owns the bus (load hex, then check memory)
6 --               CPU is held in reset so it tri-states all its outputs
7 --   loading='0' : CPU owns the bus (program execution)
8 --               TB drives 'Z' so it doesn't interfere
9 --
10 -- CPU, memory, and TB all share the same resolved bus signals.
11 -- Only one non-Z driver is active at any given time. PC relative offsets
12 -- will need to be adjusted when pipelining.
13 --
14 -- Program image (assembled; 37 words = 148 bytes):
15 --
16 -- Disassembly (address : encoding : instruction):
17 --   0x0000: DE1E  MOV.L @(0x1E,PC),R14    R14 = 0x00001000
18 --   0x0002: DF1F  MOV.L @(0x1F,PC),R15    R15 = 0x000011F0
19 --   0x0004: D11F  MOV.L @(0x1F,PC),R1     R1 = 0xDEADBEEF
20 --   0x0006: 2E12  MOV.L R1,@R14          [0x1000] = 0xDEADBEEF
21 --   0x0008: 62E2  MOV.L @R14,R2          R2 = 0xDEADBEEF
22 --   0x000A: 7E04  ADD #4,R14             R14 = 0x1004
23 --   0x000C: D11E  MOV.L @(0x1E,PC),R1    R1 = 0xAABBCCDD
24 --   0x000E: 6318  SWAP.B R1,R3           R3 = 0xAABDDCC
25 --   0x0010: 2E32  MOV.L R3,@R14         [0x1004] = 0xAABDDCC
26 --   0x0012: 7E04  ADD #4,R14
27 --   0x0014: 6419  SWAP.W R1,R4           R4 = 0xCCDDAABB
28 --   0x0016: 2E42  MOV.L R4,@R14         [0x1008] = 0xCCDDAABB
29 --   0x0018: 7E04  ADD #4,R14
30 --   0x001A: 651E  EXTS.B R1,R5           R5 = 0xFFFFFDD
31 --   0x001C: 2E52  MOV.L R5,@R14         [0x100C] = 0xFFFFFDD
32 --   0x001E: 7E04  ADD #4,R14
33 --   0x0020: 661D  EXTU.W R1,R6           R6 = 0x0000CCDD
34 --   0x0022: 2E62  MOV.L R6,@R14         [0x1010] = 0x0000CCDD
35 --   0x0024: 7E04  ADD #4,R14
36 --   0x0026: D119  MOV.L @(0x19,PC),R1    R1 = 0x12345678
37 --   0x0028: D219  MOV.L @(0x19,PC),R2    R2 = 0xABCDEF00
38 --   0x002A: 221D  XTRCT R1,R2           R2 = 0x5678ABCD
39 --   0x002C: 2E22  MOV.L R2,@R14         [0x1014] = 0x5678ABCD
40 --   0x002E: 7E04  ADD #4,R14
41 --   0x0030: D118  MOV.L @(0x18,PC),R1    R1 = 0xC0DE1234
42 --   0x0032: 1E10  MOV.L R1,@(0,R14)     [0x1018] = 0xC0DE1234
43 --   0x0034: 53E0  MOV.L @(0,R14),R3     R3 = 0xC0DE1234
44 --   0x0036: 2E32  MOV.L R3,@R14         [0x1018] confirmed
45 --   0x0038: 7E04  ADD #4,R14
46 --   0x003A: E008  MOV #8,R0             R0 = 8
47 --   0x003C: D111  MOV.L @(0x11,PC),R1    R1 = 0xDEADBEEF
48 --   0x003E: 0E16  MOV.L R1,@(R0,R14)    [0x1024] = 0xDEADBEEF
49 --   0x0040: 02EE  MOV.L @(R0,R14),R2    R2 = 0xDEADBEEF
50 --   0x0042: 2E22  MOV.L R2,@R14         [0x101C] = 0xDEADBEEF
51 --   0x0044: 7E04  ADD #4,R14
52 --   0x0046: 0018  SETT
53 --   0x0048: 0129  MOV.T R1              R1 = 1
54 --   0x004A: 2E12  MOV.L R1,@R14         [0x1020] = 0x00000001
55 --   0x004C: 7E04  ADD #4,R14
56 --   0x004E: D10E  MOV.L @(0x0E,PC),R1    R1 = 0xAABBCCDD
57 --   0x0050: 2F16  MOV.L R1,@-R15        push 0xAABBCCDD
58 --   0x0052: 62F6  MOV.L @R15+,R2        pop R2 = 0xAABBCCDD
59 --   0x0054: 2E22  MOV.L R2,@R14         [0x1024] = 0xAABBCCDD
60 --   0x0056: 7E04  ADD #4,R14
61 --   0x0058: E103  MOV #3,R1
62 --   0x005A: E204  MOV #4,R2
63 --   0x005C: 312C  ADD R2,R1             R1 = 7
64 --   0x005E: E307  MOV #7,R3
65 --   0x0060: 3130  CMP/EQ R3,R1          T = 1 (7 == 7)

```

```

66  --      0x0062: 8902  BT +2                skip to 0x0066 (T=1)
67  --      0x0064: E1FF  MOV #-1,R1            (skipped)
68  --      0x0066: 2E12  MOV.L R1,@R14        [0x1028] = 7
69  --      0x0068: 7E04  ADD #4,R14
70  --      0x006A: D107  MOV.L @(0x07,PC),R1    R1 = 0xAABBCCDD
71  --      0x006C: 621F  EXT5.W R1,R2          R2 = 0xFFFFCCDD
72  --      0x006E: 2E22  MOV.L R2,@R14        [0x102C] = 0xFFFFCCDD
73  --      0x0070: 7E04  ADD #4,R14
74  --      0x0072: A000  BRA .                  self-loop
75  --      0x0074: 0009  NOP                    (delay slot)
76  --      0x0076: 0009  NOP
77  --      -- Data constants --
78  --      0x0078: 00001000
79  --      0x007C: 000011F0
80  --      0x0080: DEADBEEF
81  --      0x0084: AABBCCDD
82  --      0x0088: 12345678
83  --      0x008C: ABCDEF00
84  --      0x0090: CODE1234
85  -----
86
87  library ieee;
88  use ieee.std_logic_1164.all;
89  use ieee.numeric_std.all;
90  use std.textio.all;
91
92  entity TB1_DataTransfer is
93  end entity;
94
95  architecture sim of TB1_DataTransfer is
96
97      constant CLK_PERIOD : time      := 20 ns;
98      constant PROG_HEX   : string    := "tb1_program.hex";
99      constant MEM_WORDS  : integer   := 256;
100
101      signal clk          : std_logic := '0';
102      signal reset        : std_logic := '0';
103      signal loading      : std_logic := '1';
104
105      -- Shared bus – CPU, memory, and TB all connect here
106      signal AB : std_logic_vector(31 downto 0);
107      signal DB : std_logic_vector(31 downto 0);
108      signal RE0, RE1, RE2, RE3 : std_logic;
109      signal WE0, WE1, WE2, WE3 : std_logic;
110
111      -- TB-side bus signals
112      signal tb_AB : std_logic_vector(31 downto 0) := (others => '0');
113      signal tb_DB : std_logic_vector(31 downto 0) := (others => '0');
114      signal tb_RE : std_logic := '1';
115      signal tb_WE : std_logic := '1';
116
117  begin
118
119      clk <= not clk after CLK_PERIOD / 2;
120
121      -- TB bus mux: drives bus when loading='1', releases ('Z') when loading='0'
122      -- During loading='0' the CPU drives the bus instead (via its port map)
123      AB <= tb_AB when loading = '1' else (others => 'Z');
124      DB <= tb_DB when (loading = '1' and tb_WE = '0') else (others => 'Z');
125      RE0 <= tb_RE when loading = '1' else 'Z';
126      RE1 <= tb_RE when loading = '1' else 'Z';
127      RE2 <= tb_RE when loading = '1' else 'Z';
128      RE3 <= tb_RE when loading = '1' else 'Z';
129      WE0 <= tb_WE when loading = '1' else 'Z';
130      WE1 <= tb_WE when loading = '1' else 'Z';
131      WE2 <= tb_WE when loading = '1' else 'Z';
132      WE3 <= tb_WE when loading = '1' else 'Z';

```

```

133
134 -- CPU connects directly to the shared bus
135 dut : entity work.SH2_CPU
136 port map (
137     Reset => reset, NMI => '1', INT => '1', clock => clk,
138     AB  => AB,   DB  => DB,
139     RE0 => RE0,  RE1 => RE1,  RE2 => RE2,  RE3 => RE3,
140     WE0 => WE0,  WE1 => WE1,  WE2 => WE2,  WE3 => WE3
141 );
142
143 mem : entity work.MEMORY32x32
144 generic map (
145     MEMSIZE      => MEM_WORDS,
146     START_ADDR0  => 16#00000000#,
147     START_ADDR1  => 16#00001000#,
148     START_ADDR2  => 16#10000000#,
149     START_ADDR3  => 16#20000000#
150 )
151 port map (
152     RE0 => RE0,  RE1 => RE1,  RE2 => RE2,  RE3 => RE3,
153     WE0 => WE0,  WE1 => WE1,  WE2 => WE2,  WE3 => WE3,
154     MemAB => AB,  MemDB => DB
155 );
156
157 stim : process
158     file      hex_file : text;
159     variable fstatus   : file_open_status;
160     variable l         : line;
161     variable word      : std_logic_vector(31 downto 0);
162     variable addr      : unsigned(31 downto 0);
163     variable pass_cnt  : integer := 0;
164     variable fail_cnt  : integer := 0;
165
166     procedure check(a : std_logic_vector(31 downto 0);
167                     exp : std_logic_vector(31 downto 0);
168                     tag : string) is
169     begin
170         tb_AB <= a;
171         tb_RE <= '0';
172         tb_WE <= '1';
173         wait for CLK_PERIOD;
174         if DB = exp then
175             pass_cnt := pass_cnt + 1;
176             report "PASS: " & tag severity note;
177         else
178             fail_cnt := fail_cnt + 1;
179             report "FAIL: " & tag
180                 & " got=0x" & to_hstring(DB)
181                 & " exp=0x" & to_hstring(exp) severity error;
182         end if;
183         tb_RE <= '1';
184         wait for CLK_PERIOD;
185     end procedure;
186
187 begin
188     -----
189     -- LOAD PHASE
190     -----
191     loading <= '1';
192     reset   <= '0';
193     tb_RE   <= '1';
194     tb_WE   <= '1';
195     wait for CLK_PERIOD * 2;
196
197     file_open(fstatus, hex_file, PROG_HEX, read_mode);
198     assert fstatus = open_ok
199         report "Cannot open " & PROG_HEX severity failure;

```

```

200
201     addr := (others => '0');
202     while not endfile(hex_file) loop
203         readline(hex_file, l);
204         if l'length > 0 then
205             hread(l, word);
206             tb_AB <= std_logic_vector(addr);
207             tb_DB <= word;
208             tb_WE <= '0';
209             wait for CLK_PERIOD;
210             tb_WE <= '1';
211             wait for CLK_PERIOD;
212             addr := addr + 4;
213         end if;
214     end loop;
215     file_close(hex_file);
216
217     -----
218     -- CPU PHASE
219     -----
220     wait until rising_edge(clk);
221     loading <= '0';
222     reset    <= '1';
223
224     wait for CLK_PERIOD * 2; -- hold reset for a couple cycles to ensure CPU is fully reset and tri-stating its
outputs
225
226     for i in 1 to 300 loop
227         wait until rising_edge(clk);
228         --wait for 0 ns;
229     end loop;
230
231     -----
232     -- CHECK PHASE
233     -----
234     wait until rising_edge(clk);
235     reset    <= '0';
236     loading <= '1';
237     tb_RE    <= '1';
238     tb_WE    <= '1';
239     wait for CLK_PERIOD * 2;
240
241     check(x"00001000", x"DEADBEEF", "MOV.L @Rm roundtrip");
242     check(x"00001004", x"AABDDCC", "SWAP.B");
243     check(x"00001008", x"CCDDAABB", "SWAP.W");
244     check(x"0000100C", x"FFFFFFDD", "EXTS.B");
245     check(x"00001010", x"0000CCDD", "EXTU.W");
246     check(x"00001014", x"5678ABCD", "XTRCT");
247     check(x"00001018", x"C0DE1234", "MOV.L @(disp,Rn) roundtrip");
248     check(x"0000101C", x"DEADBEEF", "MOV.L @(R0,Rn) indexed");
249     check(x"00001020", x"00000001", "MOVT after SETT");
250     check(x"00001024", x"AABCCDD", "pre-dec/post-inc roundtrip");
251     check(x"00001028", x"00000007", "ADD+CMPEQ+BT");
252     check(x"0000102C", x"FFFCDD", "EXTS.W");
253
254     -----
255     -- SUMMARY
256     -----
257     report "TB1: PASSED=" & integer'image(pass_cnt)
258           & " FAILED=" & integer'image(fail_cnt) severity note;
259     if fail_cnt = 0 then
260         report "*** ALL TESTS PASSED ***" severity note;
261     else
262         assert false
263         report integer'image(fail_cnt) & " TEST(S) FAILED"
264         severity failure;
265     end if;

```

```
266         wait;  
267     end process;  
268  
269 end architecture;  
270
```